MACHINE LEARNING

Module Code: CSC83101

LECTURE

TUYISHIMIRE Joseph

tuyishimirejo1992@gmail.com

UNIVERSITY Of KIGALI

Convolutional Neural Network (CNN) in Machine Learning

Convolutional Neural Networks (CNNs) are deep learning models designed to process data with a grid-like topology such as images.

They are the foundation for most modern computer vision applications to detect features within visual data.

A Convolutional Neural Network (CNN) is a specialized type of deep learning model designed particularly for processing and analyzing visual data, such as images and videos. Unlike traditional neural networks, CNNs are uniquely structured to recognize patterns and spatial hierarchies in data by mimicking how the human brain interprets visual information.

At the core of a CNN are layers that apply small, learnable filters called kernels across the input image. These filters slide over the image in a process called convolution, detecting simple features like edges, textures, and colors in early layers. As the data progresses through deeper layers, the CNN combines these simple features to identify more complex patterns, such as shapes, objects, and ultimately entire scenes

UNIVERSITY Of KIGALI

This hierarchical feature learning is adaptive and automatic, meaning the CNN adjusts the filters during training to optimize its ability to recognize relevant features.

This adjustment happens through a process known as backpropagation, where the network learns from its mistakes by minimizing the difference between its predictions and the actual labels of the training data.

Due to this architecture and learning mechanism, CNNs have become the backbone of many computer vision applications, including image classification, object detection, facial recognition, and medical image analysis, offering high accuracy and efficiency in understanding visual content

why CNNs

Traditional fully connected neural networks treat every pixel in an image as an independent feature, connecting each pixel directly to neurons in the next layer.

This approach ignores the important spatial relationships between pixels how pixels close to each other form meaningful patterns like edges, textures, or shapes. As a result, fully connected networks lose the structural information inherent in images and become inefficient when dealing with high-dimensional input like images.

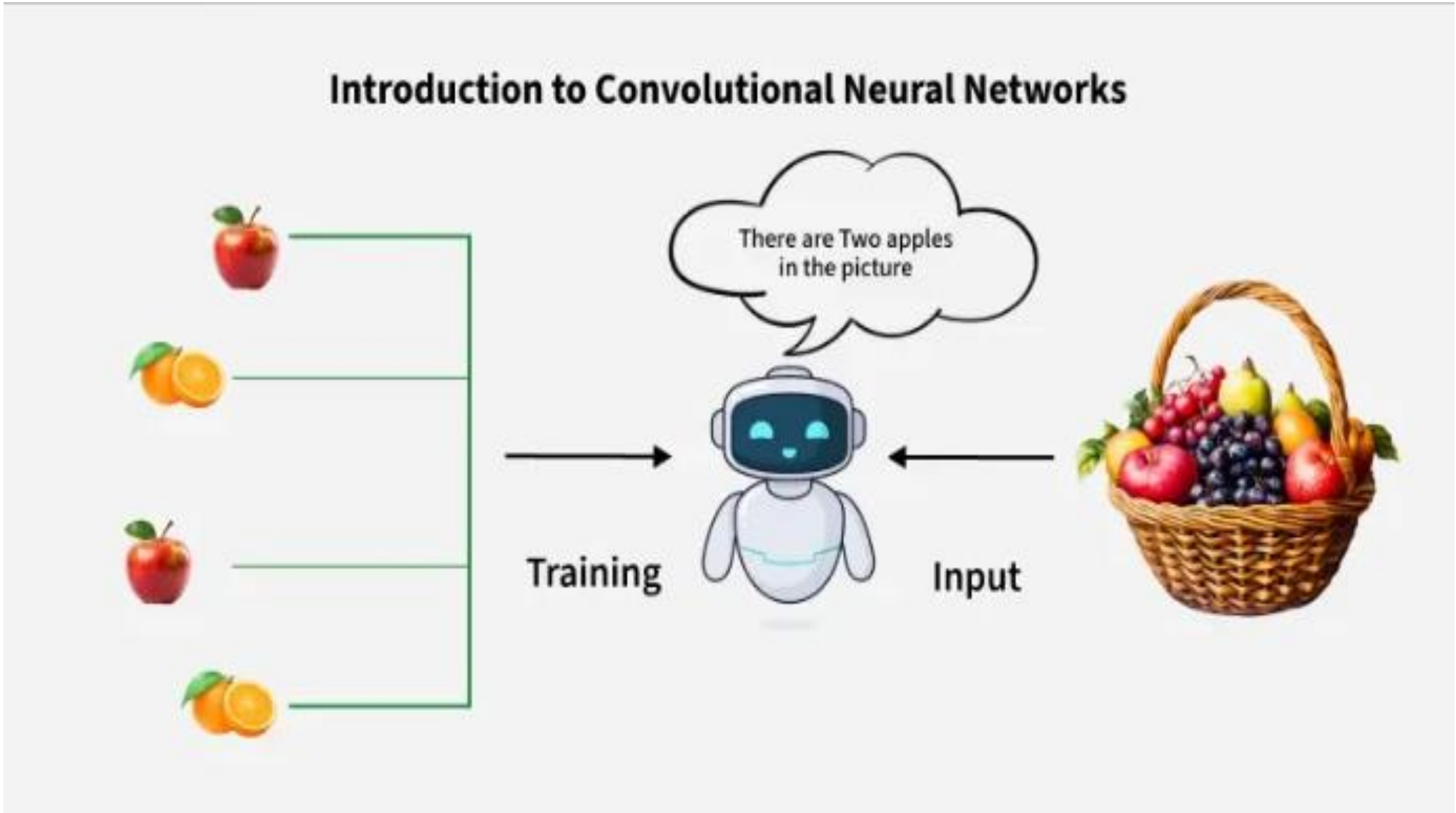
UNIVERSITY Of KIGALI

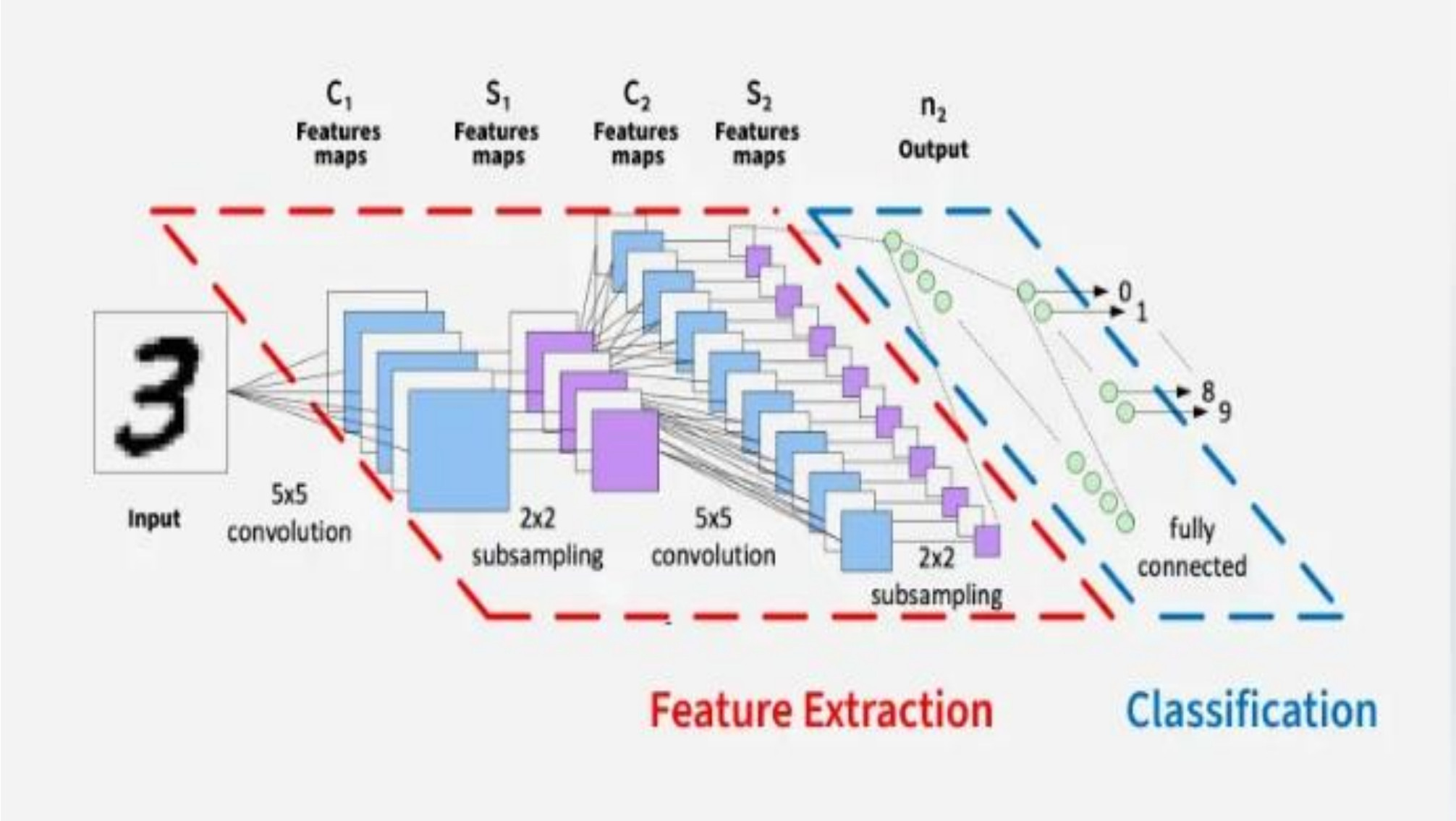
In contrast, Convolutional Neural Networks (CNNs) are designed to preserve and exploit these spatial relationships.

By using small, localized filters (kernels) that scan over regions of the image (small squares of pixels), CNNs capture local patterns and features. These learned features are then combined hierarchically to represent more complex structures in the image

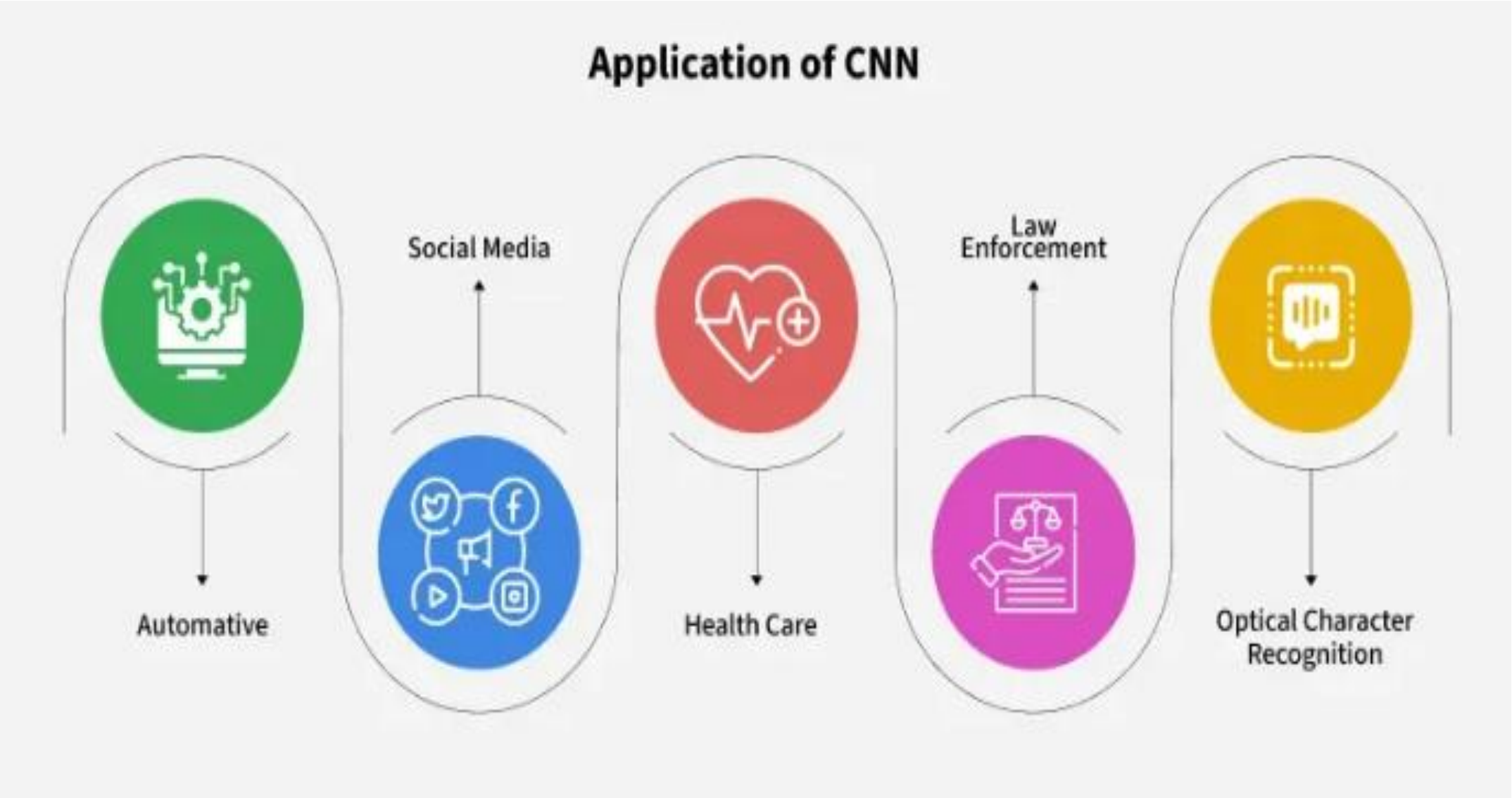
Because CNNs focus on local connectivity and weight sharing (the same filter is applied across the whole image), they significantly reduce the number of parameters compared to fully connected networks. This makes CNNs more efficient to train and better at generalizing from data. Moreover, their architecture inherently respects the 2D structure of images, enabling them to learn patterns regardless of where they appear in the imag

UNIVERSITY Of KIGALI


Introduction to Convolutional Neural Networks
There are Two apples in the picture
Training
Input

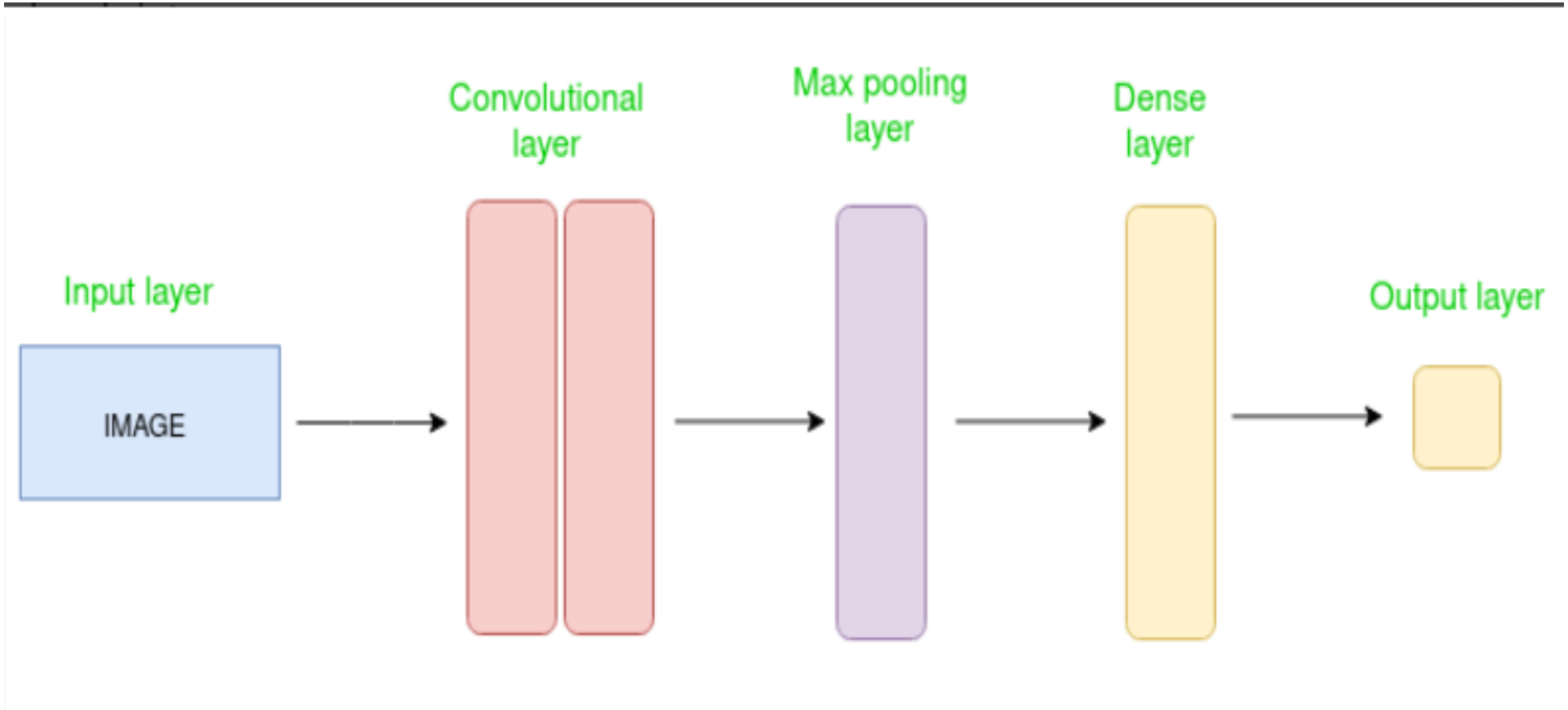


UNIVERSITY Of KIGALI



UNIVERSITY Of KIGALI

Key Concepts in CNNs



Convolutional layers

Convolutional layers are essential parts of convolutional neural networks (CNNs), which are widely used in fields like computer vision and image processing. The main function of a convolutional layer is to apply a mathematical operation called convolution to the input data, such as an image.

This convolution operation involves a small matrix called a filter or kernel. The filter slides over the input image step by step. At each position, it multiplies its values element wise with the corresponding values of the input it covers, then sums those multiplied values to produce a single number. This number forms part of a new output called a feature map.

- By repeating this process across the whole image, the convolutional layer creates a feature map that highlights specific patterns detected by the filter such as edges, textures, or shapes. These detected features are crucial for the network to understand and recognize objects within images.
- In summary, convolutional layers transform raw input images into meaningful feature maps that reveal important visual information, enabling CNNs to perform tasks like image recognition effectively.

Key Components of a Convolutional Layer

Filters (Kernels):

- Small matrices that slide over the input to extract specific features.
- Each filter detects particular patterns, such as horizontal edges, vertical edges, or textures.
- Filter values are learned and optimized during training.

Stride:

- The number of pixels the filter moves at each step while scanning the input.
- Larger strides reduce the spatial size of the output feature map and speed up computation.

Padding:

- Extra pixels (usually zeros) added around the input's border to control output size.
- Common types:
- Valid padding: No padding; output shrinks compared to input.
- Same padding: Padding applied so output size matches input size

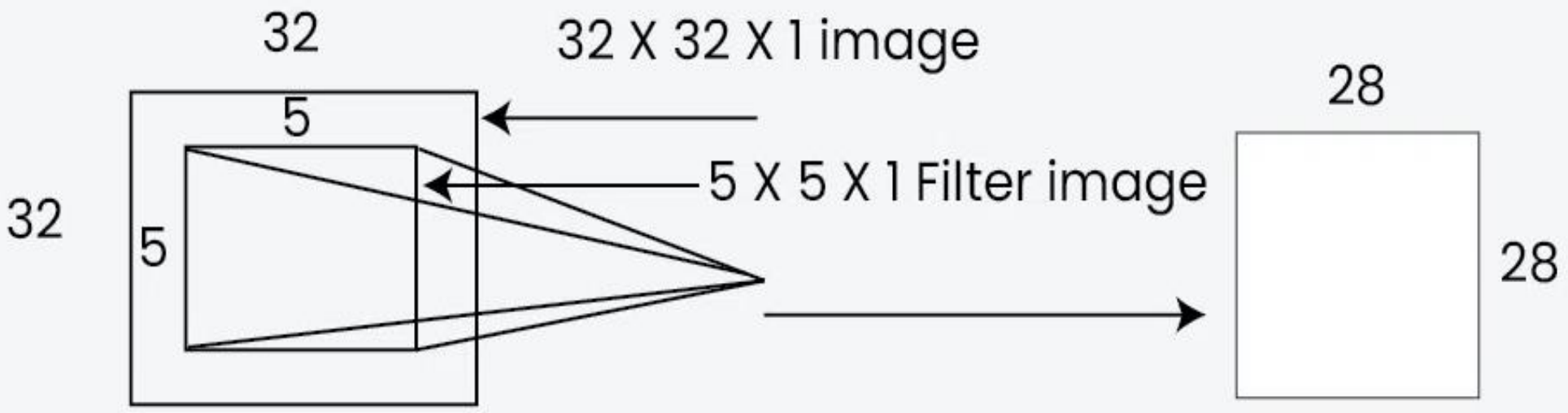
Activation Function:

- Applied after convolution to introduce non-linearity, enabling the network to learn complex features.
- Common activations include ReLU, Tanh, and Leaky ReL

Example Of Convolution Layer

Consider an input image of size 32x32x3 (32x32 pixels with 3 color channels). A convolution layer with ten 5x5 filters, a stride of 1 and 'same' padding will produce an output feature map of size 32x32x10. Each of the 10 filters detects different features in the input image.

Convolution layer



Pooling Layer

A pooling layer in a Convolutional Neural Network (CNN) plays a critical role in reducing the spatial dimensions specifically, the width and height of the input feature maps produced by convolutional layers. This dimensionality reduction serves multiple important purposes. Primarily, it helps to decrease the computational burden and memory requirements of the network, as smaller feature maps mean fewer calculations and less data to process in subsequent layers. Additionally, by reducing the number of parameters, pooling helps to mitigate the risk of overfitting, which can enhance the model's ability to generalize well to new, unseen data.

Pooling achieves this by sliding a small, typically square, window (also called a filter or kernel) over the input feature map. At each position of this window, the pooling operation summarizes the information contained within that region into a single representative value. This process effectively condenses the feature map while preserving the most salient information.

There are two widely used types of pooling operations:

- Max Pooling: This operation selects the maximum value within each window. By capturing the strongest or most prominent feature in the local region, max pooling helps highlight key patterns or activations such as edges, textures, or other distinctive features. This is particularly useful in tasks like image recognition, where the presence of a feature can be more important than its exact location.

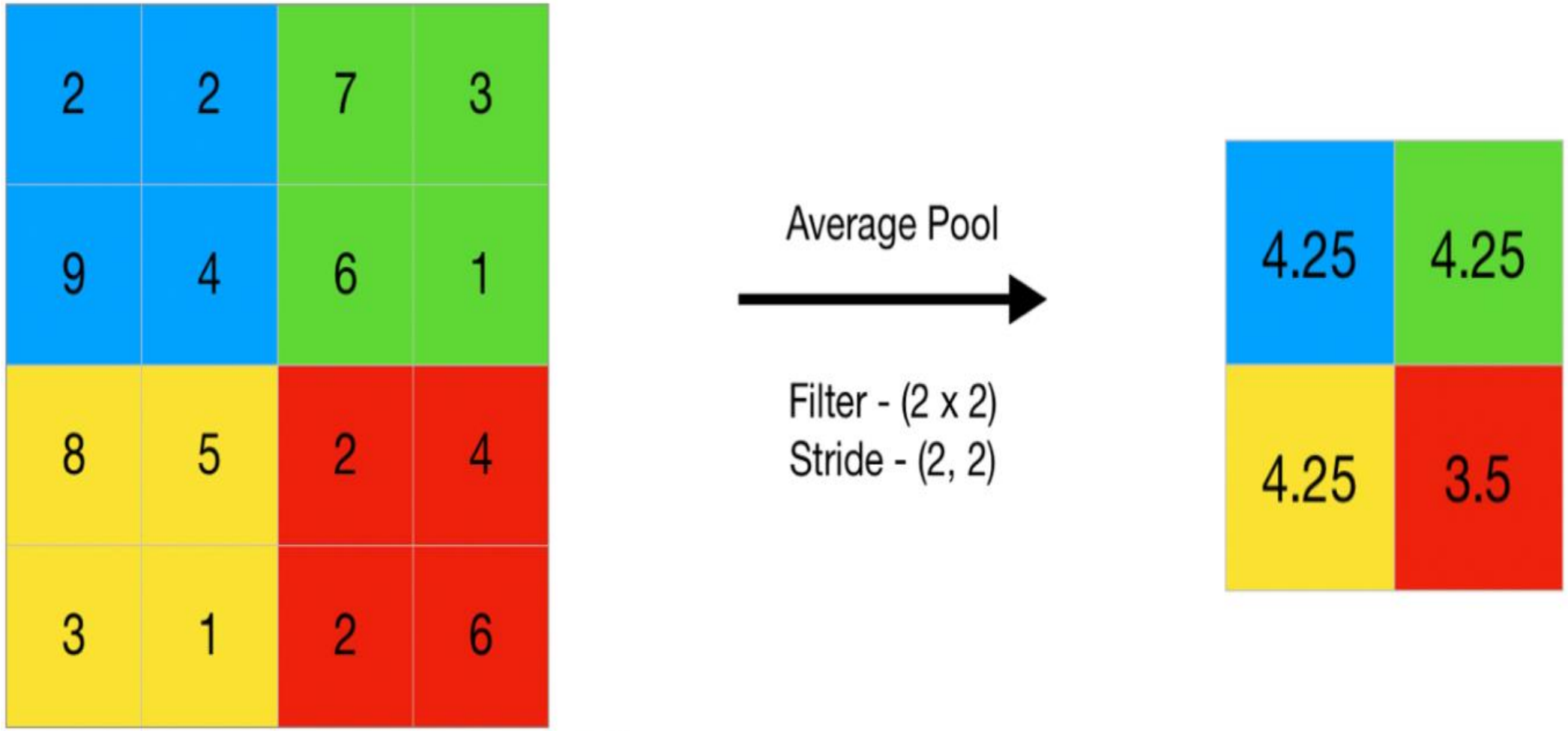
2 2 7 3
9 4 6 1
8 5 2 4
3 1 2 6
Max Pool
Filter - (2 x 2)
Stride - (2, 2)
9 7
8 6

```python
from tensorflow.keras.layers import MaxPooling2D
import numpy as np

# Example input feature map
feature_map = np.array([
    [1, 3, 2, 9],
    [5, 6, 1, 7],
    [4, 2, 8, 6],
    [3, 5, 7, 2]
]).reshape(1, 4, 4, 1)

# Applying max pooling
max_pool = MaxPooling2D(pool_size=(2, 2), strides=2)
output = max_pool(feature_map)

print(output.numpy().reshape(2, 2))
```

2. Average Pooling

Average pooling computes the average of the elements present in the region of feature map covered by the filter. Thus, while max pooling gives the most prominent feature in a particular patch of the feature map, average pooling gives the average of features present in a patch.

Average pooling provides a more generalized representation of the input. It is useful in the cases where preserving the overall context is important.


2 2 7 3
9 4 6 1
8 5 2 4
3 1 2 6
Average Pool
Filter - (2 x 2)
Stride - (2, 2)
4.25 4.25
4.25 3.5

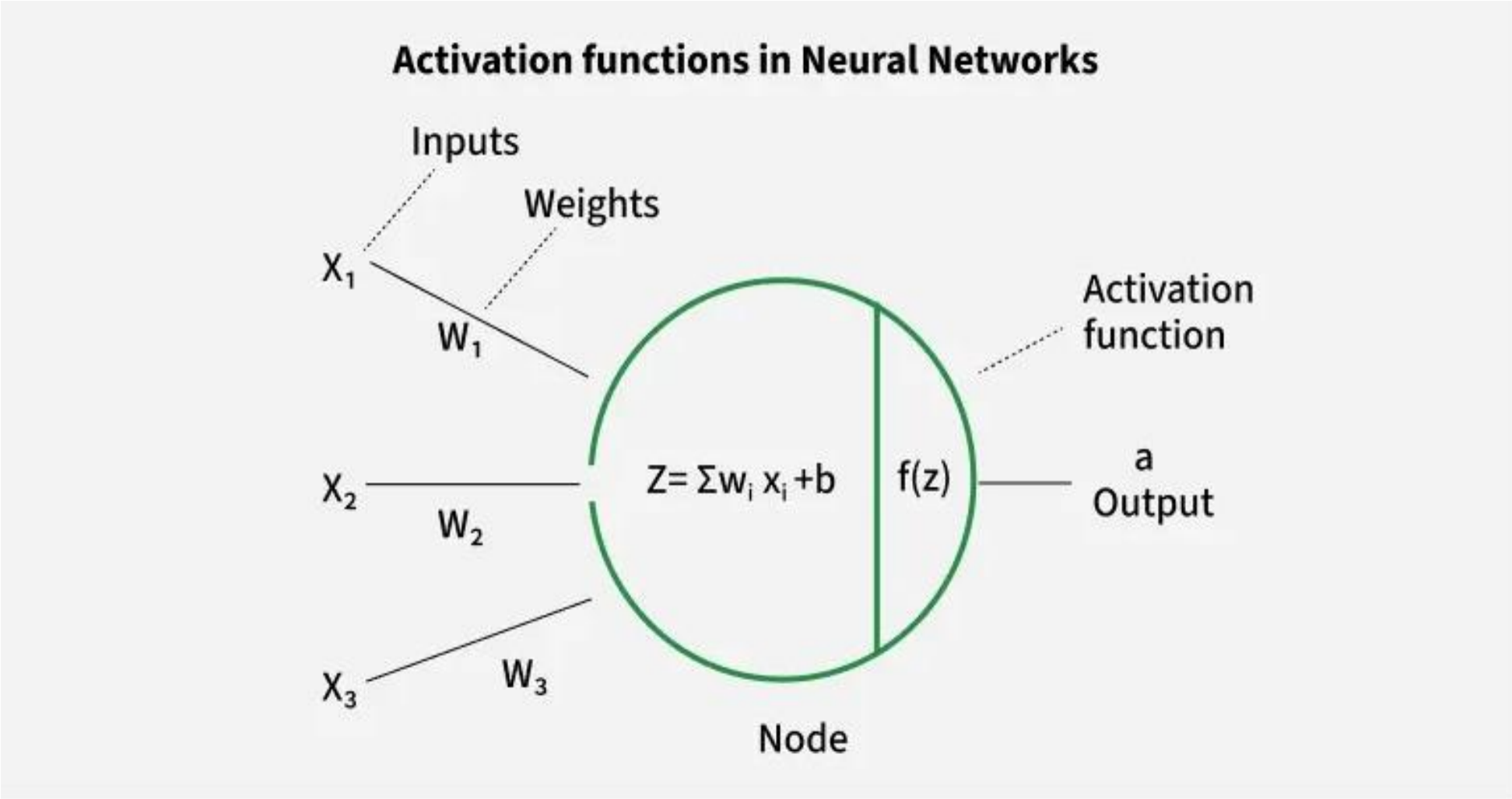
```python
import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import AveragePooling2D

feature_map = np.array([
    [1, 3, 2, 9],
    [5, 6, 1, 7],
    [4, 2, 8, 6],
    [3, 5, 7, 2]
], dtype=np.float32).reshape(1, 4, 4, 1)  # Convert to float32

# Applying average pooling
avg_pool = AveragePooling2D(pool_size=(2, 2), strides=2)
output = avg_pool(feature_map)
print(output.numpy().reshape(2, 2))
```

UNIVERSITY Of KIGALI

Activation functions in Neural Networks

- An activation function in a neural network is a mathematical function applied to the output of a neuron. It introduces non-linearity, enabling the model to learn and represent complex data patterns. Without it, even a deep neural network would behave like a simple linear regression model.
- Activation functions decide whether a neuron should be activated based on the weighted sum of inputs and a bias term. They also make backpropagation possible by providing gradients for weight updates

UNIVERSITY Of KIGALI


Activation functions in Neural Networks
Inputs
Weights
X_1
W_1
X_2
W_2
X_3
W_3
$Z= \Sigma w_i x_i +b$
f(z)
Activation function
a Output
Node

Why Non-Linearity is Important

- Real-world data is rarely linearly separable.
- Non-linear functions allow neural networks to form **curved decision boundaries**, making them capable of handling complex patterns (e.g., classifying apples vs. bananas under varying colors and shapes).
- They ensure networks can model advanced problems like image recognition, NLP and speech processing

Mathematical Example

Consider a neural network with:

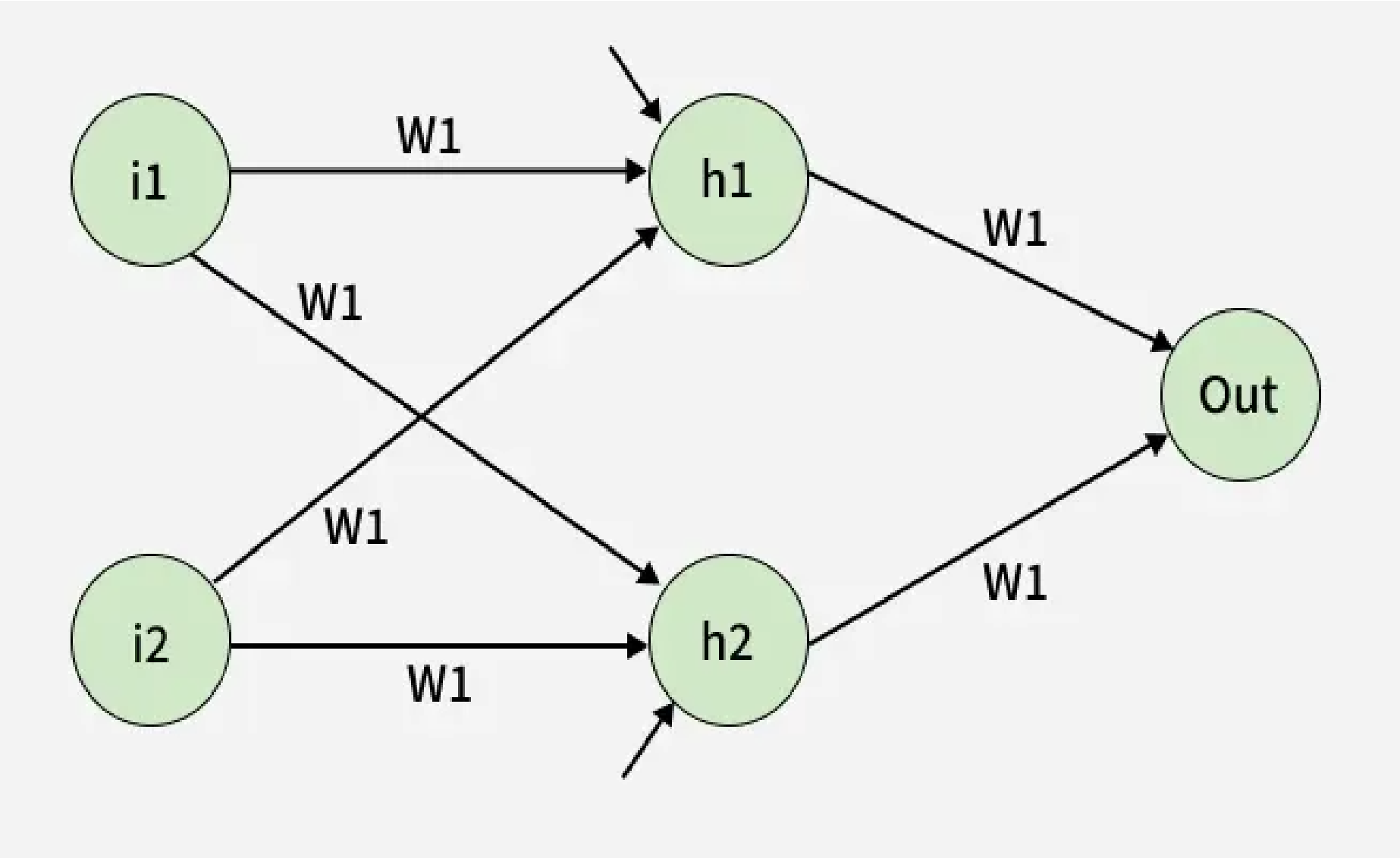
Inputs: i_1, i_2

Hidden layer: neurons h_1 and h_2

Output layer: one neuron (output)

Weights: $w_1, w_2, w_3, w_4, w_5, w_6$

Biases: b_1 for hidden layer, b_2 for output layer


i1
i2
h1
h2
Out
W1
W1
W1
W1
W1
W1

The hidden layer outputs are:

$$h_1 = i_1.w_1 + i_2.w_3 + b_1$$

$$h_2 = i_1.w_2 + i_2.w_4 + b_2$$

The output before activation is:

$$\text{output} = h_1.w_5 + h_2.w_6 + \text{bias}$$

Without activation, these are linear equations.

To introduce non-linearity, we apply a sigmoid activation:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\text{final output} = \sigma(h_1.w_5 + h_2.w_6 + \text{bias})$$

Fully Connected Layer in Deep Learning

Fully Connected (FC) layers are also known as dense layers which are used in neural networks especially in of deep learning. They are a type of neural network layer where every neuron in the layer is connected to every neuron in the previous and subsequent layers. The "fully connected" descriptor comes from the fact that each of the neurons in these layers is connected to every activation in the previous layer creating a highly interconnected network.

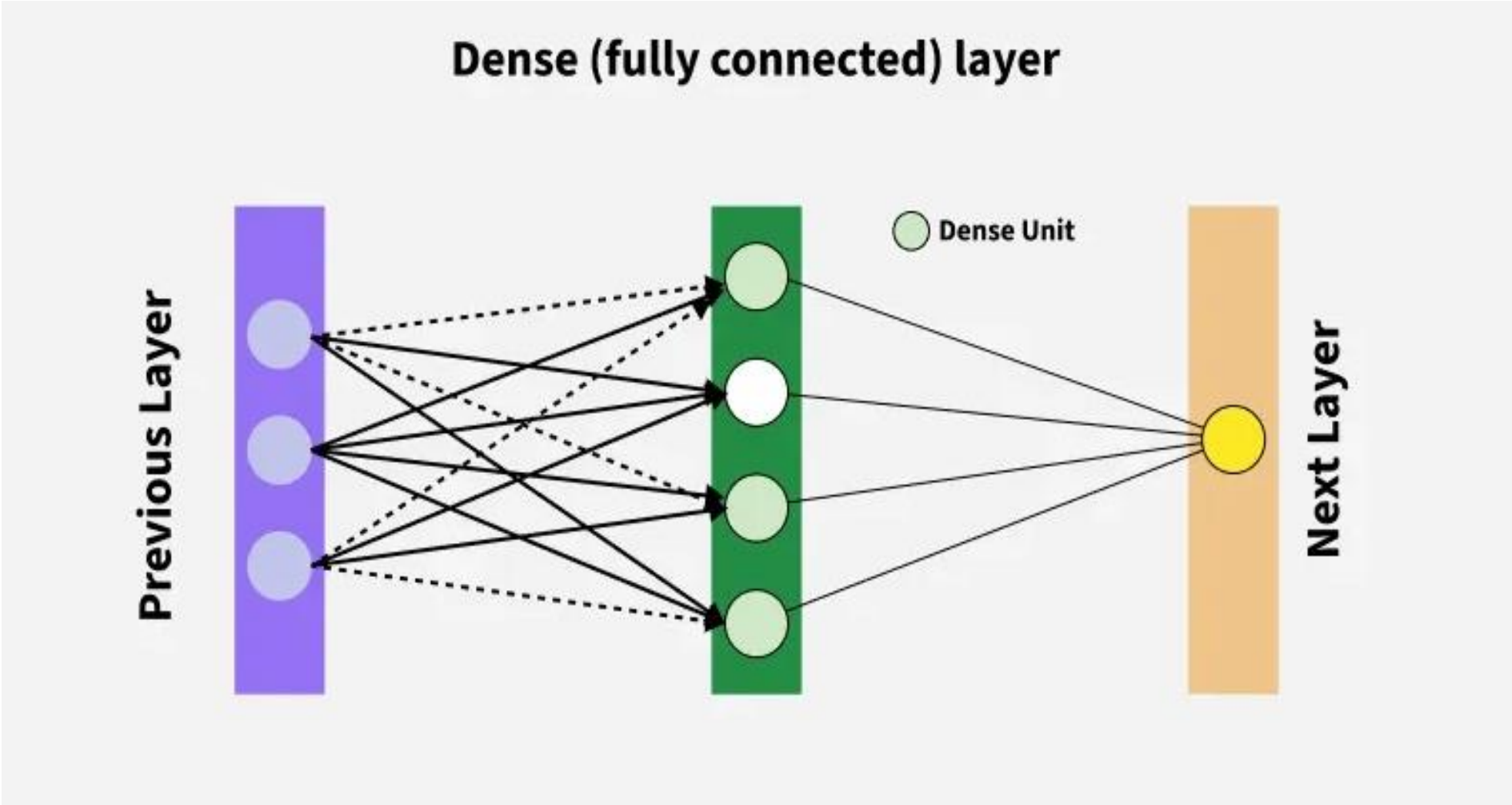
- In CNNs fully connected layers often follow convolutional and pooling layers used to interpret the feature maps generated by these layers into the final output categories or predictions.

- In fully connected feedforward networks these layers are the main building blocks that directly process the input data into outputs.

Structure of Fully Connected Layers

The structure of FC layers is one of the most significant factors that define how it works in a neural network.

This structure involves the fact that every neuron in one layer will interconnect with every neuron in the subsequent layer.


Dense (fully connected) layer
Previous Layer
Dense Unit
Next Layer

Key Components of Fully Connected Layers

A Fully Connected layer is characterized by its dense interconnectivity. Here's a breakdown of its key components:

- Neurons: Basic units that receive inputs from all neurons of the previous layer and send outputs to all neurons of the subsequent layer.
- Weights: Each connection between neurons has an associated weight indicating the strength and influence of one neuron on another.
- Biases: A bias term for each neuron helps adjust the output along with the weighted sum of inputs.
- Activation Function: Functions like ReLU, Sigmoid or Tanh introduce non-linearity to the model helping it to learn complex patterns and behaviors.

Working of CNN

Input Image: CNN receives an input image which is preprocessed to ensure uniformity in size and format.

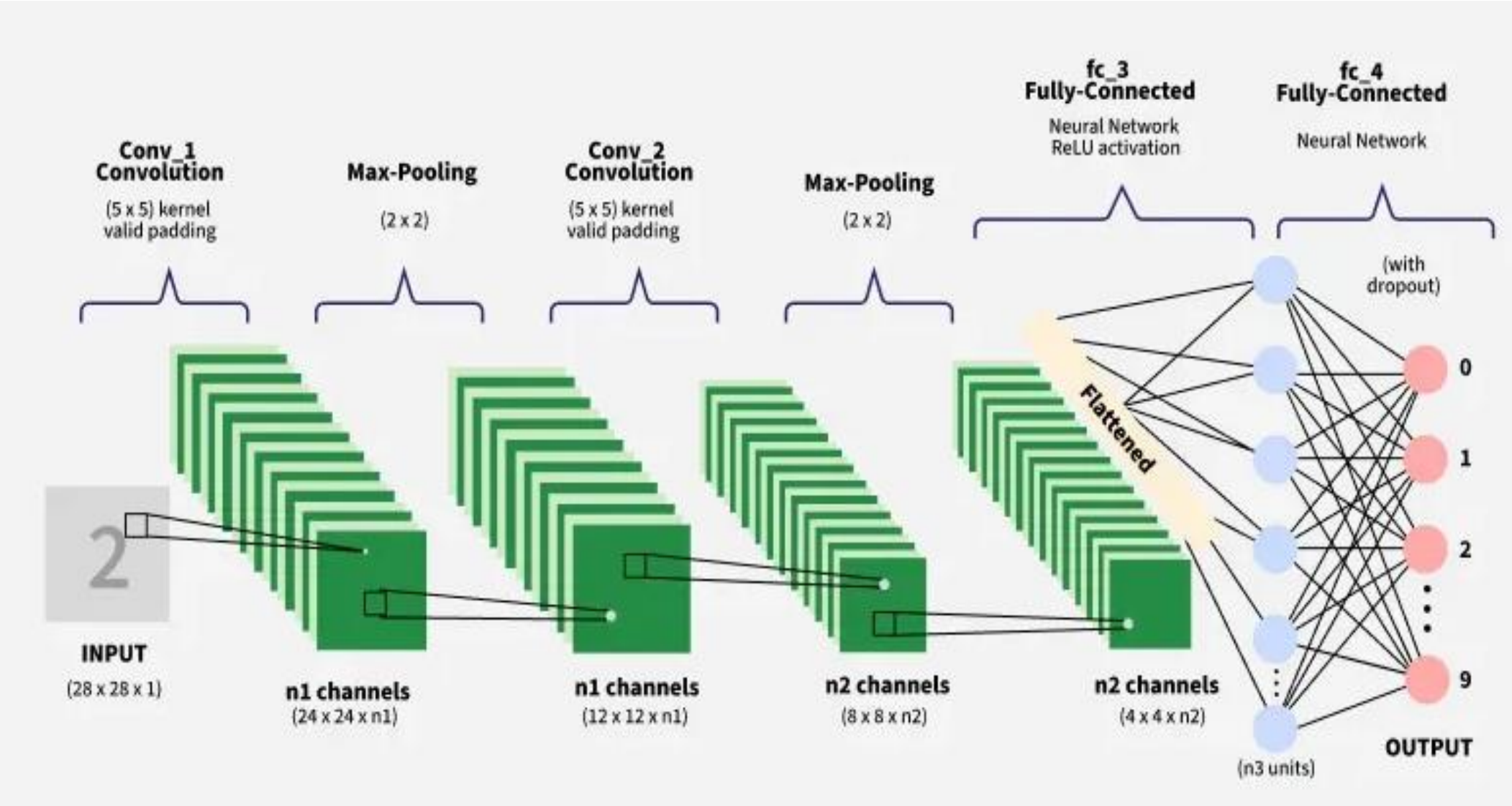
Convolutional Layers: Filters are applied to the input image to extract features like edges, textures and shapes.

Pooling Layers: The feature maps generated by the convolutional layers are downsampled to reduce dimensionality.

Fully Connected Layers: The downsampled feature maps are passed through fully connected layers to produce the final output, such as a classification label.

Output: The CNN outputs a prediction, such as the class of the image.

UNIVERSITY Of KIGALI


Conv_1
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
Conv_2
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
fc_3
Fully-Connected
Neural Network
ReLU activation
fc_4
Fully-Connected
Neural Network
(with
dropout)
Flattened
2
INPUT
(28 x 28 x 1)
n1 channels
(24 x 24 x n1)
n1 channels
(12 x 12 x n1)
n2 channels
(8 x 8 x n2)
n2 channels
(4 x 4 x n2)
(n3 units)
0
1
2
9
OUTPUT

Training a Convolutional Neural Network

CNNs are trained using a supervised learning approach. This means that the CNN is given a set of labeled training images. The CNN learns to map the input images to their correct labels.

The training process for a CNN involves the following steps:

- **Data Preparation:** The training images are preprocessed to ensure that they are all in the same format and size.
- **Loss Function**: A loss function is used to measure how well the CNN is performing on the training data. The loss function is typically calculated by taking the difference between the predicted labels and the actual labels of the training images.
- **Optimizer**: An optimizer is used to update the weights of the CNN in order to minimize the loss function.
- **Backpropagation:** Backpropagation is a technique used to calculate the gradients of the loss function with respect to the weights of the CNN. The gradients are then used to update the weights of the CNN using the optimizer.

UNIVERSITY Of KIGALI

How to Evaluate CNN Models

Efficiency of CNN can be evaluated using a variety of criteria. Among the most popular metrics are:

- Accuracy: Accuracy is the percentage of test images that the CNN correctly classifies.
- Precision: Precision is the percentage of test images that the CNN predicts as a particular class and that are actually of that class.
- Recall: Recall is the percentage of test images that are of a particular class and that the CNN predicts as that class.
- F1 Score: The F1 Score is a harmonic mean of precision and recall. It is a good metric for evaluating the performance of a CNN on classes that are imbalanced.

Different Types of CNN Models

- **LeNet**: One of the earliest successful CNNs, developed in the late 1990s for handwritten digit recognition on the MNIST dataset. It laid the foundation for modern CNNs.
- **AlexNet**: The first CNN to win the ImageNet Challenge in 2012. It features five convolutional layers, three pooling layers, and three fully connected layers, pioneering deep CNN architectures.
- **ResNet**: Introduces skip connections to enable training of very deep networks by learning residual functions, reducing overfitting and improving performance on complex image tasks.

- GoogleNet (InceptionNet): Achieves high accuracy with fewer parameters by learning features at multiple scales simultaneously using the Inception module, optimizing both performance and efficiency.

- VGG: Known for its deep and uniform architecture using small 3x3 convolution filters stacked in many layers. Variants like VGG-16 and VGG-19 showed strong performance on ImageNet, emphasizing the power of depth.

Applications

- Image classification: CNNs are the state-of-the-art models for image classification. They can be used to classify images into different categories such as cats and dogs.
- Object detection: It can be used to detect objects in images such as people, cars and buildings. They can also be used to localize objects in images which means that they can identify the location of an object in an image.
- Image segmentation: It can be used to segment images which means that they can identify and label different objects in an image. This is useful for applications such as medical imaging and robotics.

Video analysis: It can be used to analyze videos such as tracking objects in a video or detecting events in a video. This is useful for applications such as video surveillance and traffic monitoring.

Advantages

- High Accuracy: They can achieve high accuracy in various image recognition tasks.
- Efficiency: They are efficient, especially when implemented on GPUs.
- Robustness: They are robust to noise and variations in input data.
- Adaptability: It can be adapted to different tasks by modifying their architecture.

Disadvantages

- Complexity: It can be complex and difficult to train, especially for large datasets.
- Resource-Intensive: It require significant computational resources for training and deployment.
- Data Requirements: They need large amounts of labeled data for training.
- Interpretability: They can be difficult to interpret making it challenging to understand their predictions.

CNN Model and Its Implementation Using Python

A Convolutional Neural Network (CNN) is a deep learning model primarily used for image processing, recognition, and classification. It automatically detects important features without any human supervision.

Step 1: Import Required Libraries

- import tensorflow as tf
- from tensorflow.keras.models import Sequential
- from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
- from tensorflow.keras.datasets import mnist
- from tensorflow.keras.utils import to_categorical

- **import tensorflow as tf**

Imports TensorFlow, a powerful open-source library for machine learning and deep learning. It provides the core functionalities to build and train neural networks.

- **from tensorflow.keras.models import Sequential**

Imports the Sequential model class from Keras (the high-level API of TensorFlow). Sequential allows you to build a neural network layer-by-layer in a linear stack.

from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

- **Imports various neural network layers:**
- Conv2D: Applies 2D convolution filters to extract spatial features from images.
- MaxPooling2D: Performs down sampling by taking the maximum value over patches, reducing spatial dimensions and computation.
- Flatten: Converts multi-dimensional inputs into a 1D vector to connect convolutional layers with dense layers.
- Dense: A fully connected layer where each neuron is connected to all neurons in the previous layer.
- Dropout: A regularization technique that randomly deactivates a fraction of neurons during training to reduce overfitting.

UNIVERSITY Of KIGALI

from tensorflow.keras.datasets import mnist

Imports the MNIST dataset, a popular dataset of 28x28 grayscale images of handwritten digits (0-9), commonly used for training and testing image classification models.

- **from tensorflow.keras.utils import to_categorical**

Imports the utility function to_categorical which converts integer class labels (e.g., 3) into one-hot encoded vectors (e.g., [0,0,0,1,0,0,0,0,0,0]) suitable for multi-class classification.

Step 2: Load and Prepare Dataset

You load the dataset and prepare it for the CNN. The images must be reshaped to include a channel dimension (e.g., grayscale has 1 channel), normalized to scale pixel values between 0 and 1 for better training, and the labels converted to one-hot encoding for classification.

Step 2: Load Dataset

General Syntax:

(x_train, y_train), (x_test, y_test) =<dataset_name>.load_data()

Example:

(x_train, y_train), (x_test, y_test) = mnist.load_data()

Explanation:

Load the dataset into training and testing sets.

x_train, x_test: input images.

y_train, y_test: corresponding labels.

Step 3: Preprocess Data

General Syntax:

 # Reshape input to include channel dimension (e.g., grayscale = 1)

```
x_train = x_train.reshape(-1, height, width, channels).astype('float32') / scale_factor
x_test = x_test.reshape(-1, height, width, channels).astype('float32') / scale_factor
```

 # One-hot encode labels for classification tasks

```
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)
```

Explanation:

- Reshape images to 4D tensors: (samples, height, width, channels).
- Normalize pixel values to [0,1] by dividing by 255.
- Convert integer labels to categorical one-hot vectors.

Example:

```
x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

Step 4: Build CNN Model

General Syntax:

```
model = Sequential([
    Conv2D(filters, kernel_size, activation='activation_function',
input_shape=(height, width, channels)),
    MaxPooling2D(pool_size=(pool_height, pool_width)),
    ... # additional Conv2D/Pooling layers
    Flatten(),
    Dense(units, activation='activation_function'),
    Dense(num_classes, activation='softmax')
])
```

UNIVERSITY Of KIGALI

Explanation:

CNN layers extract spatial features.

MaxPooling reduces spatial dimensions.

Flatten converts 2D feature maps to 1D vectors.

Dense layers perform classification.

Final layer uses softmax for multi-class probabilities.

```
model = models.Sequential([
    layers.Conv2D(16, (3,3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(32, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(2, activation='softmax')  # 2 classes: small_square and large_square
])

model.summary()
```

Step 5: Compile Model

General Syntax:

```
model.compile(optimizer='optimizer_name',
              loss='loss_function',
              metrics=['metric_name'])
```

Explanation:

- Define optimizer for training (e.g., Adam).
- Specify loss function appropriate for classification.
- Set evaluation metrics (accuracy here).

Example:

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

Step 6: Train Model

General Syntax:

```
history = model.fit(x_train, y_train,
                    validation_data=(x_val, y_val),
                    epochs=num_epochs,
                    batch_size=batch_size)
```

Explanation:

- Train model on training data.
- Validate on test/validation data.
- Specify number of epochs and batch size.
- Store training history for visualization.

Example:

```
history = model.fit(x_train, y_train,
                    validation_data=(x_test, y_test),
                    epochs=5,
                    batch_size=64)
```

Step 7: Evaluate Model

General Syntax:

```
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=2)
print(f'Test accuracy: {test_accuracy}')
```

Explanation:

- Evaluate model performance on unseen test data.
- Print accuracy or other metrics.

Example:

```
test_loss, test_acc = model.evaluate(x_test, y_test,
verbose=2)
print(f'\nTest accuracy: {test_acc:.4f}')
```

Step 8: Plot Training & Validation Metrics

General Syntax:

```
import matplotlib.pyplot as plt
plt.figure(figsize=(width, height))
plt.subplot(1,2,1)
plt.plot(history.history['metric_name'])
plt.plot(history.history['val_metric_name'])
plt.title('Training and Validation Metric')
plt.xlabel('Epoch')
plt.ylabel('Metric')
plt.legend(['Train', 'Validation'])
plt.subplot(1,2,2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
```

UNIVERSITY Of KIGALI

Explanation:

- Visualize how accuracy and loss change over epochs.
- Compare training and validation performance.

```
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

Step 9 (Optional): Make Predictions and Visualize

General Syntax:

```
predictions = model.predict(x_test)
def plot_image(i, predictions_array, true_labels, images):
    # visualize image i with predicted and true labels
plt.figure(figsize=(width, height))
for i in range(num_images):
    plt.subplot(rows, cols, i+1)
    plot_image(i, predictions, y_test, x_test)
plt.tight_layout()
plt.show()
```

UNIVERSITY Of KIGALI

Explanation:

- Predict labels on test set.
- Plot sample images with predicted and true labels.
- Color-code correct (green) or incorrect (red) prediction

```
import numpy as np
predictions = model.predict(x_test)
def plot_image(i, predictions_array, true_labels, images):
    predictions_array, true_label, img = predictions_array[i],
np.argmax(true_labels[i]), images[i].reshape(28,28)
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(img, cmap=plt.cm.binary)
    predicted_label = np.argmax(predictions_array)
    color = 'green' if predicted_label == true_label else 'red'
```

```
plt.xlabel(f"Pred: {predicted_label} ({100*np.max(predictions_array):.2f}%)\nTrue: {true_label}", color=color)
plt.figure(figsize=(10,4))
for i in range(10):
    plt.subplot(2,5,i+1)
    plot_image(i, predictions, y_test, x_test)
plt.tight_layout()
plt.show()
```